

Assignment: Decoding of Received Signal

Each student has been provided with a received signal file in the form of a NumPy array (.npy), named according to their roll number. The task is to decode the transmitted message from this received signal by following the described procedure.

How the Received Signal File Was Created

- 1 A phrase in English (including alphabets, spaces, and punctuation marks) was selected. Each phrase ends with a full stop (.) and has a length between 45 and 60 characters (including spaces and punctuation).
- 2 Each character in the phrase was converted to its ASCII equivalent, then each ASCII number was converted into a 7-bit binary representation. This forms the complete binary message sequence.
- 3 The binary bits were modulated using BPSK, with the mapping: bit 0 to -1 and bit 1 to +1.
- 4 A 13-length Barker code was pre-appended to the BPSK modulated sequence to enable frame synchronization at the receiver.
- 5 The Barker-appended BPSK signal was pulse-shaped using a Raised-Cosine filter with parameters: $\text{sps} = 8$, Number of filter taps = 101, Roll-off factor (β) = 0.35.
- 6 The pulse-shaped signal was transmitted through an AWGN channel where white Gaussian noise was added.
- 7 To mimic real-world conditions, the following distortions were added:
 - Frequency Offset: Between 8 kHz – 10 kHz (assuming a sampling rate of 1 MHz).
 - Frame Offset: The pulse-shaped signal was tiled multiple times, and the received samples were collected starting from a random index between 100 and 1000.

The final received signal was saved as a .npy file and distributed individually to students. (Each student has a unique phrase, frequency offset, and frame offset.)

Steps to Decode the Received Signal

- 1 Load the .npy file corresponding to your roll number and store the samples in a NumPy array.
- 2 Matched Filtering.
- 3 Use the PSD to estimate and identify the frequency offset.
- 4 Coarse Frequency Synchronization.
- 5 Fine Frequency Synchronization (Optional).
- 6 Perform downsampling by taking every sps -th sample to obtain time sync output.
- 7 Frame Synchronization using Barker code and correlation.
- 8 BPSK Demodulation.
- 9 Bit-to-Text Conversion: Group the demodulated bits into 7-bit segments. Convert each 7-bit group to its ASCII equivalent. Convert ASCII values to characters to reconstruct the decoded text.
- 10 Since the receiver does not know the message length, collect characters until the first full stop (.) appears. This represents the decoded phrase.
- 11 Do normalization wherever required in the above steps.

Note: Once you obtain the decoded phrase, fill it in the Google Form provided. Also, upload your Python code file used for decoding in the Google Classroom.

Marks will be awarded based on:

- **Accuracy:** Number of correctly decoded characters matching the original phrase.
- **Completeness:** Proper implementation of the decoding steps.
- **Individuality:** Each student's file contains a unique phrase and offset parameters — do not copy results from others.